

Summarize CivicFlow Plan

Contents

01  CivicFlow-AI — Day 3 Complete Interview Notes T... RESPONSE

CivicFlow-AI — Day 3 Complete Interview Notes

Today was a **major AI/GenAI milestone**. You moved from a normal FastAPI + PostgreSQL backend into an **AI-powered incident intelligence pipeline**.

Your Day 3 work covered:

```
text

AI / GenAI Fundamentals
↓
LLM + Prompt Engineering
↓
Structured JSON Output
↓
Pydantic Validation
↓
Embeddings
↓
pgvector
↓
Cosine Similarity
↓
Semantic Duplicate Detection
↓
RAG Retrieval
↓
RAG + LLM Answer Generation
```

↓
Duplicate-aware RAG

Your preparation plan also places GenAI fundamentals around LLMs, tokens, prompts, temperature and API usage, followed by structured outputs/prompt engineering and then RAG fundamentals. `filecite turn7file0`

1. Artificial Intelligence — AI

What is AI?

Artificial Intelligence is the broader field of building systems that can perform tasks that normally require intelligent behavior.

Examples:

- Understanding text
- Recognizing images
- Making predictions
- Classifying information
- Generating content
- Making recommendations

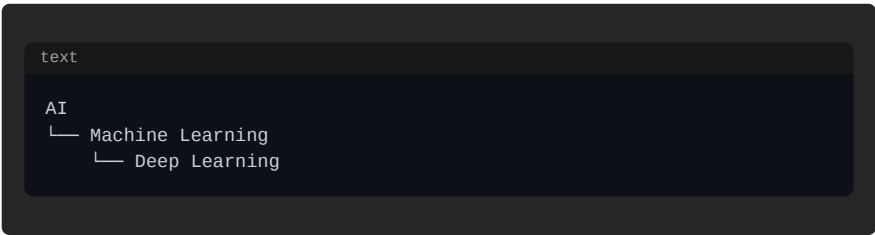
Interview answer

“Artificial Intelligence is a broad field of computer science focused on creating systems capable of performing tasks that normally require human-like intelligence, such as reasoning, classification, prediction and language understanding.”

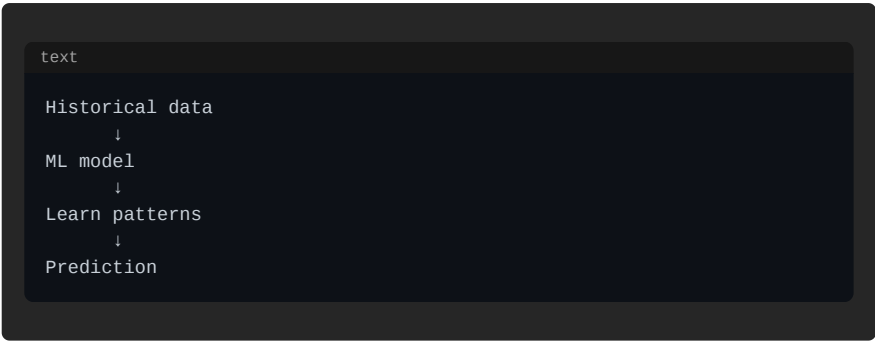
2. Machine Learning — ML

Machine Learning is a **subset of AI**.

Instead of explicitly programming every rule, an ML system learns patterns from data.



Example:



Interview answer

“Machine Learning is a subset of AI in which algorithms learn patterns from data and use those learned patterns to make predictions or decisions.”

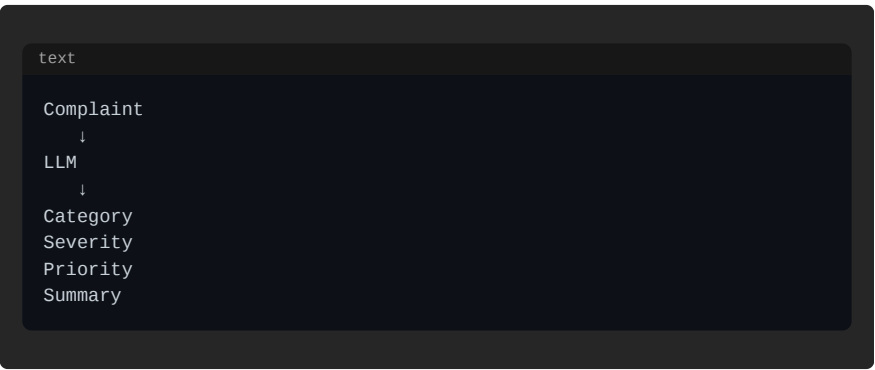
3. Generative AI

Generative AI focuses on **generating new content**.

It can generate:

- Text
- Code
- Images
- Summaries
- Structured JSON
- Answers

Your CivicFlow project uses Generative AI for:



Interview answer

“Generative AI refers to AI systems capable of generating new content based on patterns learned from their training data.”

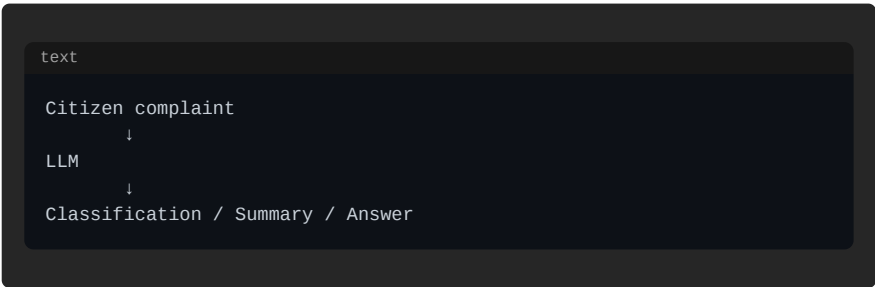
4. LLM

What is an LLM?

LLM = Large Language Model

It is a model trained on large amounts of text and designed to understand and generate natural language.

In CivicFlow:



Interview question

What is an LLM?

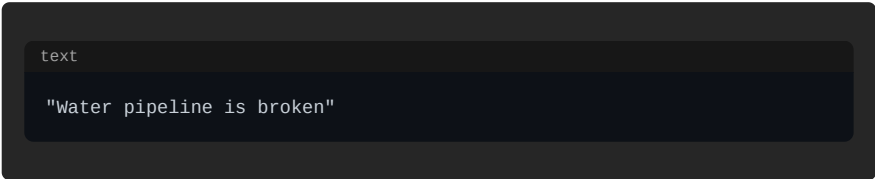
“An LLM is a large-scale language model trained on large amounts of textual data to understand and generate natural-language content.”

5. Tokens

An LLM doesn't necessarily process text as complete words.

Text is divided into **tokens**.

For example:



is converted by a tokenizer into a sequence of tokens.

Tokens can represent:

- Complete words
- Parts of words
- Punctuation
- Other text units

Why are tokens important?

They affect:

- Input size
- Output size
- Context limits
- API usage/cost

Interview question

What is a token?

“A token is a unit of text processed by a language model. Depending on the tokenizer, a token can represent a complete word, part of a word, punctuation, or another text fragment.”

6. Prompt

A **prompt** is the instruction/input given to an AI model.

Example from CivicFlow:

```
text

You are CivicFlow-AI,
a civic incident classification assistant.

Analyze the citizen complaint.

Return:
category
severity
priority
summary
```

The complaint is then provided to the model.

7. Prompt Engineering

Prompt engineering means designing instructions that guide an LLM toward the desired output.

A good prompt usually defines:

```
text

Role
↓
Task
```

```
↓
Rules
↓
Allowed values
↓
Input
↓
Expected output
```

Your CivicFlow approach

```
text

Role:
Civic incident classification assistant

Task:
Analyze complaint

Categories:
Water
Lighting
Road
Dust

Severity:
Low
Medium
High
Critical

Priority:
Low
Medium
High
Urgent
```

This constrains the model.

Interview answer

“Prompt engineering is the process of designing clear and structured instructions for an LLM so that it produces outputs appropriate for a specific task.”

8. Temperature

Temperature controls the randomness of generated output.

Conceptually:

```
text

Lower temperature
↓
More predictable output

Higher temperature
↓
More varied output
```

For structured classification, predictable output is generally desirable.

Interview answer

“Temperature is a generation parameter that influences how much variation or randomness the model uses when selecting output tokens.”

9. LLM API

Instead of running the LLM locally, your application calls an AI provider's API.

Your architecture:

```
text

FastAPI
↓
AI Service
↓
Gemini API
↓
LLM
↓
Response
↓
FastAPI
```

Your AI service isolates the AI provider logic from your router.

This is good software architecture.

10. Structured Output

This was one of the most important Day 3 concepts.

Instead of allowing the LLM to return:

```
text

The complaint seems serious and concerns water...
```

you require a structured response:

```
json
{
  "category": "Water",
  "severity": "High",
  "priority": "Urgent",
  "summary": "Water pipeline has burst near Ward 5."
}
```

Your Pydantic schema:

```
python
class AIReportResponse(BaseModel):
    category: str
    severity: str
    priority: str
    summary: str
```

Why structured output?

Because your backend can reliably consume the AI result.

```
text
LLM
↓
JSON
↓
Pydantic validation
↓
FastAPI
↓
Database
```

Interview answer

“Structured output forces an LLM to return data in a predefined format such as JSON. In CivicFlow, Pydantic validates that output before it is used by the backend.”

11. Why Pydantic is important with AI

An LLM is probabilistic.

Your backend expects predictable data.

Therefore:

```
text
LLM output
↓
Pydantic
↓
Validation
```

For example, your application expects:

```
text
severity = Low | Medium | High | Critical
```

If the model returns an unexpected structure, validation can reject it.

Interview question

Why not directly trust the LLM response?

“Because LLMs generate probabilistic outputs. I use structured output and Pydantic validation to enforce the expected application schema before using the response.”

12. AI Classification

Your CivicFlow classification system takes a citizen complaint and determines:

```
text
Category
Severity
Priority
Summary
```

Example:

```
text
Input:
"The water pipeline near Ward 5 has burst
and water is flooding the road."
```

Possible structured result:

```
json
```

```
{
  "category": "water",
  "severity": "High",
  "priority": "Urgent",
  "summary": "Burst water pipeline causing flooding near Ward 5."
}
```

13. Embeddings

This was another major Day 3 topic.

What is an embedding?

An embedding is a **vector representation of data**, commonly text, in a numerical space where semantic relationships can be measured.

Your flow:

```
text
Text
↓
Embedding model
↓
Vector
```

Your CivicFlow embedding:

```
text
```

```
Complaint
↓
Gemini embedding model
↓
768-dimensional vector
```

You successfully generated:

```
text

Dimensions: 768
```

14. Why use embeddings?

Suppose we have:

```
text

Report A:
"Water pipeline near Ward 5 has burst."

Report B:
"Water is leaking from the pipe around Ward 5."
```

The strings are different.

Normal string comparison:

```
text

Different
```

Embedding comparison:

```
text

Similar meaning
```

That's why embeddings are useful for:

- Semantic search
- Similarity search
- Duplicate detection
- Recommendation
- RAG

15. Vector

A vector is a numerical representation.

Conceptually:

```
text

Text
↓
```

```
[0.0143, -0.0091, 0.0231, ...]
```

Your CivicFlow vectors have:

```
text

768 dimensions
```

You store them in PostgreSQL using **pgvector**.

16. pgvector

`**pgvector**` allows PostgreSQL to store and search vector embeddings.

Your architecture:

```
text

PostgreSQL
+
pgvector
↓
Embedding storage
↓
Similarity search
```

Your database contains:

```
text

reports
├ id
├ title
├ description
├ location
├ category
├ severity
├ priority
├ summary
└ embedding VECTOR(768)
```

You successfully verified:

```
text

embedding_exists = true
dimensions = 768
```

17. Cosine Similarity / Cosine Distance

This is extremely important for interviews.

We need to compare two vectors.

One common measure is **cosine similarity**.

Conceptually:

```
text
```

```
Similarity ↑  
↓  
Meaning is more similar
```

Your pgvector implementation uses **cosine distance**.

Important:

```
text  
  
Smaller cosine distance  
=   
More similar
```

Your actual tests:

```
text  
  
Water complaint  
Distance = 0.0804
```

and:

```
text  
  
Road complaint  
Distance = 0.2182
```

18. Similarity Threshold

You configured:

```
python  
  
SIMILARITY_THRESHOLD = 0.20
```

Your logic:

```
python  
  
distance <= threshold
```

means potential duplicate.

Example:

```
text  
  
0.0804 <= 0.20  
↓  
True
```

Therefore:

```
text

is_duplicate = True
```

For:

```
text

0.2182 > 0.20
```

the result was:

```
text

is_duplicate = False
```

Very important interview point

Do **not** say:

"0.20 is the universal standard."

Say:

"0.20 is the initial application threshold I configured and tested against representative examples. It can be tuned based on evaluation results and false-positive/false-negative behavior."

19. Semantic Similarity

Semantic similarity means comparing the **meaning** of text rather than exact wording.

Example:

```
text

A:
"Water leaking near school."

B:
"Pipe burst outside school."
```

Different words, potentially similar meaning.

Embedding:

```
text

A → Vector A
B → Vector B
```

Then:

```
text

Vector A
  ↓
Cosine distance
  ↓
Vector B
```

20. Exact Duplicate vs Semantic Duplicate

Exact duplicate

```
text

A = "Water pipeline burst near Ward 5."

B = "Water pipeline burst near Ward 5."
```

String comparison can detect this.

Semantic duplicate

```
text

A = "Water pipeline burst near Ward 5."

B = "There is water leaking from the pipeline around Ward 5."
```

The strings are different.

Embedding similarity can identify their semantic relationship.

Interview answer

“Exact duplicate detection compares text directly, whereas semantic duplicate detection converts text into embeddings and compares their vector similarity.”

21. Your Duplicate Detection API

You implemented:

```
text

POST /api/similarity/check
```

Request:

```
json

{
  "complaint": "There is a water pipeline leak near Ward 5."
}
```

Result:

```
json

{
  "is_duplicate": true,
  "report_id": 10,
  "title": "Broken Water Pipeline",
  "distance": 0.0804,
  "threshold": 0.2
}
```

This demonstrates:

```
text
```

```
API
↓
Embedding
↓
pgvector
↓
Similarity search
↓
Threshold
↓
Duplicate decision
```

22. RAG

RAG = **Retrieval-Augmented Generation**

Instead of asking the LLM to answer only from its learned knowledge:

```
text
Question
↓
LLM
↓
Answer
```

we first retrieve relevant application data.

```
text
Question
↓
Embedding
```

```
↓
Vector Search
↓
Relevant Reports
↓
Context
↓
LLM
↓
Answer
```

23. RAG Pipeline

Your CivicFlow implementation is:

```
text
User Question
↓
Generate Query Embedding
↓
pgvector Similarity Search
↓
Retrieve Relevant Reports
↓
Build Context
↓
Gemini
↓
Grounded Answer
↓
Sources
```

This is one of the most important things to remember for interviews.

24. RAG Components

1. Query

User asks:

```
text
Are there any water pipeline problems near Ward 5?
```

2. Query embedding

Convert the question into a vector.

3. Retrieval

Search report embeddings.

4. Context

Build relevant report information:

```
text
Report ID
Title
Description
Category
Severity
Priority
Distance
```

5. Generation

Send the context + question to the LLM.

6. Answer

The LLM generates a natural-language response.

25. Why RAG?

Suppose your database contains:

```
text
Report 10:
Broken Water Pipeline
Ward 5
```

A general LLM doesn't automatically know your private CivicFlow database.

RAG gives the LLM the relevant application data.

```
text
Database
↓
Retrieve
↓
Context
↓
LLM
```

Interview answer

“RAG combines information retrieval with generative AI. The system retrieves relevant data from an external knowledge source and provides it to the LLM as context so the generated answer is grounded in that retrieved information.”

26. RAG Context

Your context builder creates information like:

```
text

Report ID: 10
Title: Broken Water Pipeline
Description: The water pipeline near Ward 5 has burst...
Category: Water
Severity: High
Priority: Urgent
Similarity distance: 0.1638
```

Then multiple reports are combined:

```
text

Report 10
---
Report 1
---
Report 2
---
...
```

This becomes the LLM's context.

27. Grounded Generation

Your RAG prompt specifically instructed the model to:

- Use only CivicFlow report information
- Avoid inventing incidents
- Avoid inventing locations
- Avoid inventing severity/priority
- Say when available context isn't sufficient

This is called **grounding**.

Interview answer

“Grounding means constraining the generated answer using trusted retrieved information instead of allowing the model to freely invent facts.”

28. Hallucination

A hallucination occurs when an AI model produces information that isn't supported by the available evidence.

Example:

Database:

```
text

No report about a bridge collapse.
```

Bad answer:

```
text

A bridge collapse was reported in Ward 7.
```

That's unsupported.

Your RAG prompt attempts to reduce this by telling the model:

```
text

Use only the provided CivicFlow context.
```

29. RAG Source Retrieval

Your API returns:

```
json

"sources": [
  {
    "report_id": 10,
    "title": "Broken Water Pipeline",
```

```
"distance": 0.1637
}
]
```

This is useful because the user can see **which reports contributed to the answer**.

This makes the system more transparent than returning only:

```
text

"Yes, there is a problem."
```

30. Duplicate-Aware RAG

Initially your retrieval returned:

```
text

Report 10
Report 7
Report 8
Report 9
```

All were essentially the same water pipeline report.

That creates unnecessary repeated context.

You improved the retrieval:

```
text

Retrieve more candidates
↓
Remove exact duplicate descriptions
↓
Keep representative results
↓
Return top unique reports
```

Your latest result contained:

```
text

10 → Broken Water Pipeline → 0.1638
1 → Updated Streetlight Issue → 0.3469
2 → Broken Streetlight → 0.3585
6 → Road pond → 0.4221
4 → Broken Road → 0.4763
```

This demonstrates an important RAG concept:

“The nearest result is not necessarily the only useful result, and duplicate retrieval can reduce context quality.”

31. Your RAG API

You implemented:

```
text
```

```
POST /api/rag/ask
```

Request:

```
json

{
  "question": "Are there any water pipeline problems near Ward 5?"
}
```

Response:

```
json

{
  "question": "Are there any water pipeline problems near Ward 5?",
  "answer": "Yes, according to the available CivicFlow reports...",
  "sources": [
    {
      "report_id": 10,
      "title": "Broken Water Pipeline",
      "distance": 0.1637
    }
  ]
}
```

HTTP:

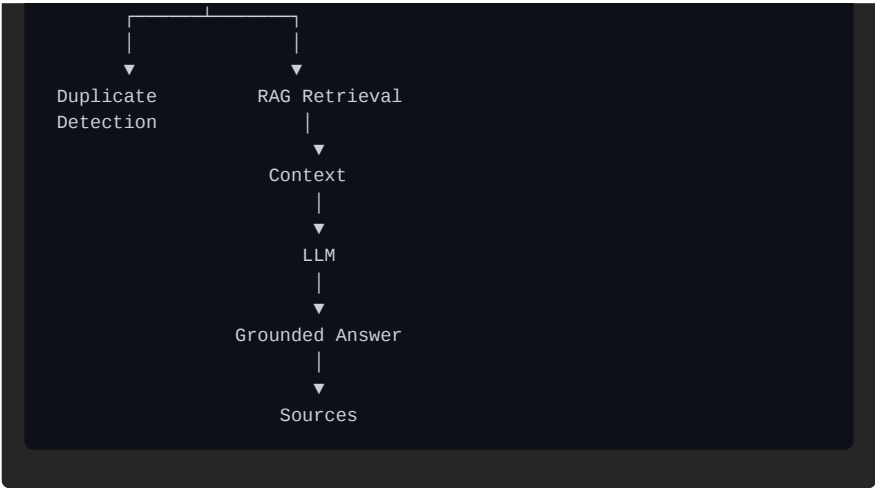
```
text

200 OK
```

So your complete RAG API is working.

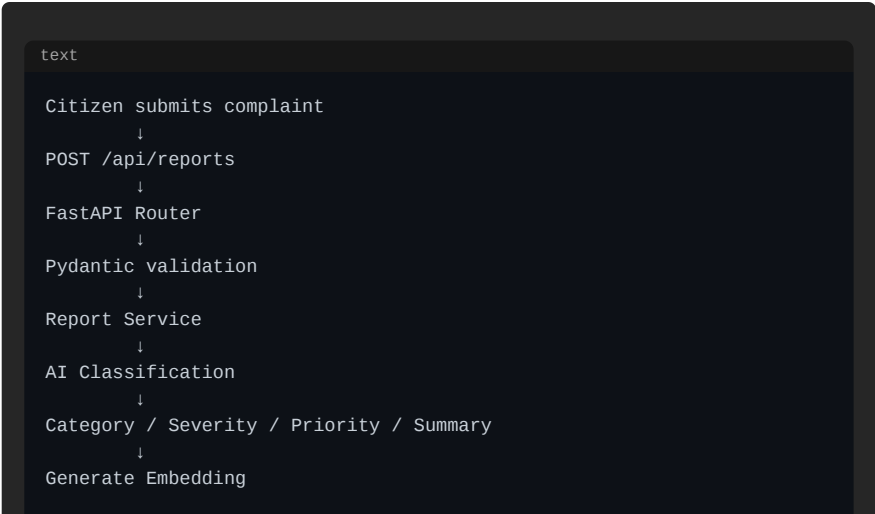
32. Complete CivicFlow AI Architecture

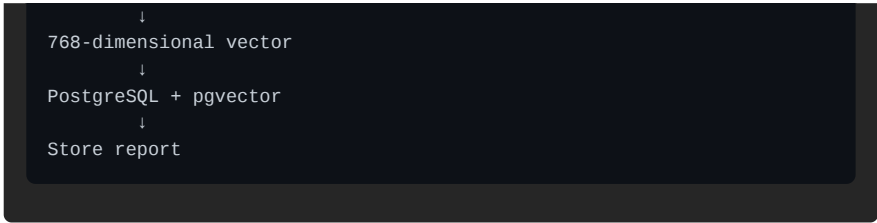
You should memorize this architecture for interviews:



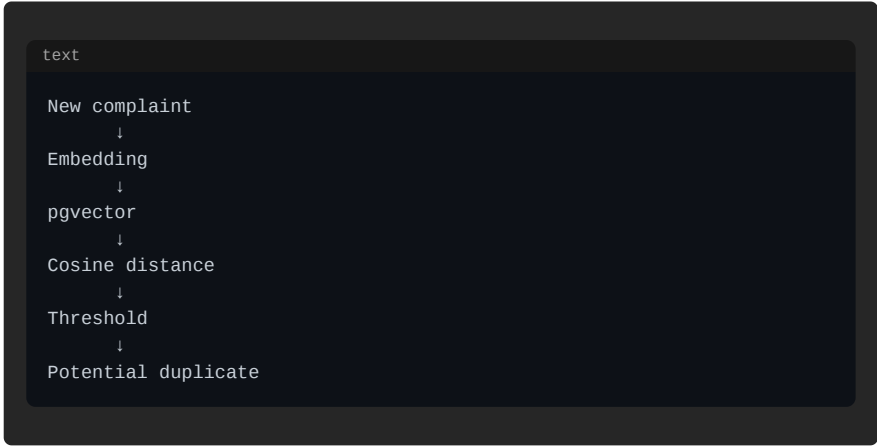
33. End-to-End CivicFlow AI Flow

For a new report:

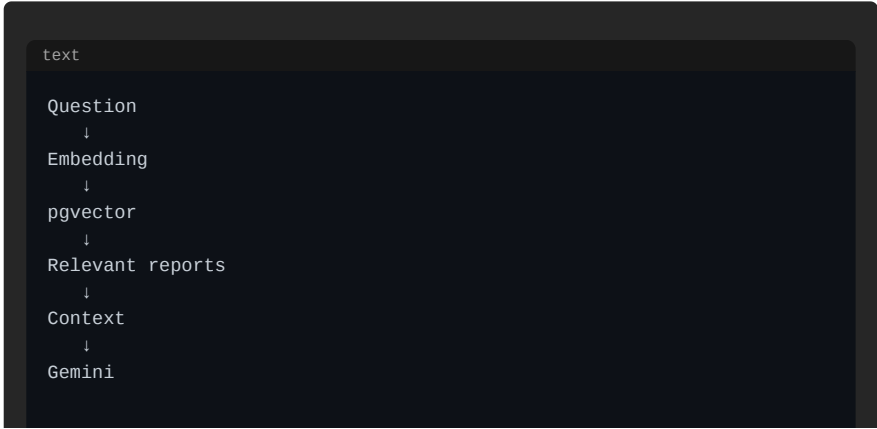




For duplicate detection:



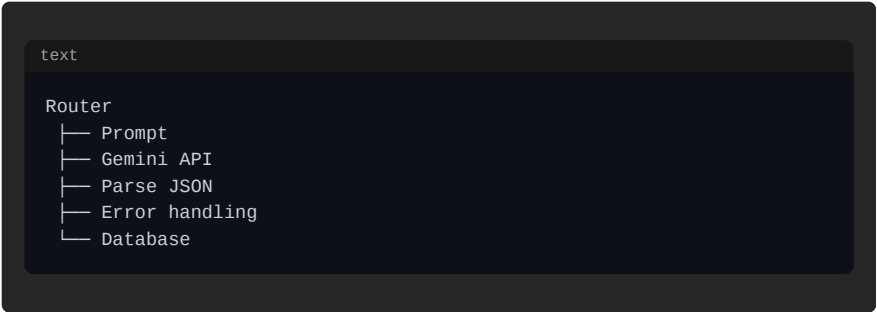
For RAG:



34. Why Separate `ai\_service.py`?

You separated AI logic from the router.

Instead of:



you have:



Interview answer

"I separated the AI integration into its own service so that the API layer remains responsible for HTTP handling while the AI service handles prompt construction, model invocation and AI response processing."

35. Why Separate Embedding Service?

You created:

```
text
embedding_service.py
```

with:

```
python
generate_embedding(text)
```

This provides a reusable abstraction.

Multiple features can use it:

```
text
Report creation
  ↓
Embedding
```

```
Duplicate detection
  ↓
Embedding
RAG
  ↓
Embedding
```

Without duplicating the Gemini embedding code.

36. Error Handling in AI

Your Gemini integration also includes handling for provider failures.

Conceptually:

```
text
FastAPI
  ↓
Gemini
  ↓
Provider failure
  ↓
HTTP 503
```

Response:

```
text
AI service is temporarily unavailable.
```

```
Please try again later.
```

Interview answer

“External AI APIs can fail independently of my application, so I handle provider errors and return an appropriate service-unavailable response rather than allowing an unhandled exception to propagate.”

37. Important Difference: LLM vs Embedding Model

This is a common interview question.

LLM

Used for:

```
text

Generation
Classification
Summarization
Question answering
```

Embedding model

Used for:

```
text

Vector representation
Semantic similarity
Search
Duplicate detection
Retrieval
```

Your CivicFlow system uses both:

```
text

LLM
↓
Classification + RAG answer

Embedding model
↓
Similarity + Duplicate detection + RAG retrieval
```

38. Common Interview Questions

Q1. What is AI?

“AI is the broader field of creating systems capable of performing tasks that normally require intelligent behavior.”

Q2. What is Generative AI?

“Generative AI produces new content such as text, code, images or structured outputs.”

Q3. What is an LLM?

“A Large Language Model is trained on large amounts of text to understand and generate natural language.”

Q4. What is a prompt?

“A prompt is the input or instruction provided to an AI model.”

Q5. What is prompt engineering?

“It is the process of designing effective instructions that guide an AI model toward the desired output.”

Q6. What are tokens?

“Tokens are the units of text processed by an AI model.”

Q7. What is temperature?

“Temperature controls the degree of randomness or variation in generated output.”

Q8. What is structured output?

“Structured output forces the model response into a predefined format such as JSON.”

Q9. Why use Pydantic with LLM output?

“To validate that the AI-generated structured data matches the schema expected by the application.”

Q10. What is an embedding?

“An embedding is a numerical vector representation of data that captures useful semantic relationships.”

Q11. Why use embeddings?

“They allow semantic comparison and retrieval based on meaning rather than exact text matching.”

Q12. What is pgvector?

“pgvector is a PostgreSQL extension that supports storing and searching vector embeddings.”

Q13. What is cosine distance?

“It measures the angular difference between vectors; smaller cosine distance means greater similarity.”

Q14. How does CivicFlow detect duplicates?

“It generates an embedding for the new complaint, searches existing report embeddings using cosine distance, and compares the nearest result against a configured threshold.”

Q15. What is semantic similarity?

“Semantic similarity measures how similar two pieces of text are in meaning rather than just comparing their exact words.”

Q16. What is RAG?

“Retrieval-Augmented Generation combines retrieval from an external knowledge source with LLM generation.”

Q17. Why use RAG?

“To provide the LLM with relevant application-specific information and ground its answer in retrieved data.”

Q18. What is hallucination?

“Hallucination is when an AI model generates information that isn't supported by the available evidence.”

Q19. How does CivicFlow reduce hallucination?

“It retrieves relevant CivicFlow reports and instructs the LLM to answer using only the provided context.”

Q20. Why return RAG sources?

“Sources provide transparency and allow the application to show which reports were used to generate the answer.”

39. Advanced Interview Questions

Q21. Why not use string matching for duplicate detection?

“String matching only detects identical or nearly identical text. Embeddings allow CivicFlow to detect semantically similar complaints even when they use different wording.”

Q22. Why store embeddings in PostgreSQL?

“CivicFlow already uses PostgreSQL, and pgvector allows vector storage and similarity search within the same database ecosystem.”

Q23. Why retrieve more candidates than the final RAG limit?

“Because several of the nearest results may be duplicates. Retrieving additional candidates gives the system enough results after duplicate filtering.”

Q24. Why isn't the closest report automatically a duplicate?

“Similarity and duplication are different concepts. A report can be the closest available result while still being below the application's duplicate threshold.”

Your actual example proves this:

```
text

Distance = 0.2182
Threshold = 0.20
```

Therefore:

```
text

Closest report ≠ duplicate
```

Q25. Why should a similarity threshold be evaluated?

“Because the appropriate threshold depends on the embedding model, domain and data. It should be validated using representative examples and adjusted based on false positives and false negatives.”

40. 60-Second CivicFlow Interview Explanation

If the interviewer says:

"Explain your CivicFlow AI project."

Say:

“CivicFlow-AI is an AI-powered civic incident intelligence system. Citizens submit complaints through the application, and the FastAPI backend validates and processes them. I use an LLM to classify complaints into category, severity and priority and generate a summary using structured JSON output validated with Pydantic.

For duplicate detection and semantic search, I generate embeddings for complaints and store them as 768-dimensional vectors in PostgreSQL using pgvector. I use cosine distance to identify semantically similar reports.

I also implemented RAG. When a user asks a question, the question is converted into an embedding, relevant civic reports are retrieved from pgvector, and those reports are provided as context to the LLM. The API returns both the generated answer and the retrieved source reports.

I also added duplicate-aware retrieval so repeated reports don't unnecessarily dominate the RAG context.”

41. Your Day 3 Achievement

Before Day 3

```
text

FastAPI
PostgreSQL
CRUD
```

After Day 3

```
text

FastAPI
+
PostgreSQL
+
Gemini LLM
+
Structured Output
+
Pydantic
+
Embeddings
+
pgvector
+
Cosine Similarity
+
Semantic Duplicate Detection
+
RAG
+
Source Retrieval
+
Duplicate-aware Retrieval
```

That's a **substantial AI backend foundation** for CivicFlow-AI.

Day 3 Quick Revision Sheet

Memorize this:

```
text

AI
→ Broad field of intelligent systems

ML
→ Learns patterns from data

GenAI
→ Generates new content

LLM
→ Language generation/understanding

Token
→ Unit processed by model

Prompt
→ Instruction/input to model

Temperature
→ Controls output randomness

Structured Output
→ Predictable JSON/schema

Pydantic
→ Validates AI output

Embedding
→ Text → numerical vector

Vector
→ Numerical representation
```

```
pgvector
→ Vector storage/search in PostgreSQL

Cosine Distance
→ Smaller = more similar

Semantic Similarity
→ Meaning-based comparison

Duplicate Detection
→ Embedding + similarity + threshold

RAG
→ Retrieve → Context → Generate

Grounding
→ Answer based on retrieved information

Hallucination
→ Unsupported/generated false information

Sources
→ Evidence retrieved for RAG answer
```

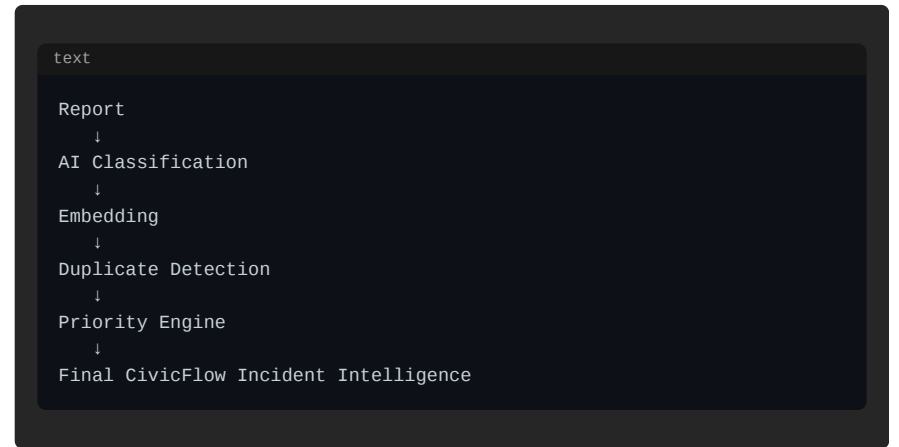
Day 3 Status

Topic	Status
AI fundamentals	✓
ML vs AI	✓
Generative AI	✓
LLM	✓
Tokens	✓

Topic	Status
Prompts	✓
Prompt engineering	✓
Temperature	✓
Gemini API	✓
Structured output	✓
JSON responses	✓
Pydantic AI validation	✓
Embeddings	✓
768-dimensional vectors	✓
PostgreSQL	✓
pgvector	✓
Cosine distance	✓
Semantic similarity	✓
Duplicate detection	✓
Similarity threshold	✓
RAG	✓
RAG retrieval	✓
RAG context	✓
LLM-generated RAG answer	✓
RAG sources	✓
Duplicate-aware RAG	✓
Git milestone commits	✓

🎯 Day 3 is complete.

Your next session is **Day 4 — Priority Engine + AI pipeline integration**, where we'll connect:



That will take the individual AI components you've built and turn them into one integrated CivicFlow workflow.